

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

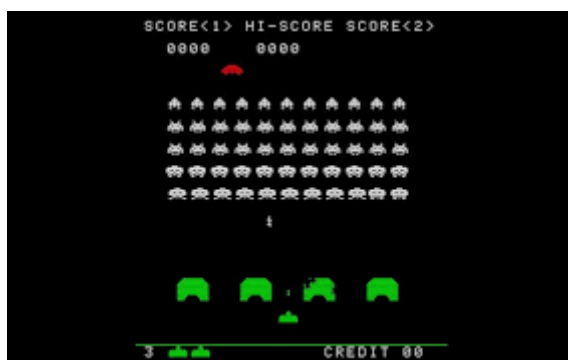
Especificação do Trabalho

Space Invaders

Introdução

Este trabalho tem como objetivo avaliar, de forma prática, os conceitos ensinados no curso de Programação I. A meta deste trabalho é implementar uma versão simplificada do clássico jogo de Arcade chamado *Space Invaders*¹ seguindo as especificações descritas neste documento e utilizando os conceitos ensinados em sala para manter boas práticas de programação.

O jogo consiste em um desafio de um jogador, no qual o mesmo controla uma nave espacial que se move horizontalmente em um mapa bidimensional enquanto naves alienígenas se aproximam da base da tela. O objetivo do jogo é eliminar todos os inimigos antes que todas as vidas acabem ou os aliens cheguem à base da tela. Uma vida é perdida toda vez que a nave é acertada por um tiro inimigo.



Descrição do Trabalho

Neste trabalho, você deverá implementar um jogo similar ao *Space Invaders*. O programa deverá rodar no terminal, assim como todos os outros exercícios feitos em sala. De maneira geral, o programa é iniciado via linha de comando (terminal) e realiza a leitura do estado inicial do jogo, que inclui a definição do mapa e das configurações das naves.

Com o jogo inicializado, serão executados movimentos dados pelo usuário na entrada padrão. Durante a execução, o programa apresentará os estados parciais na saída padrão até o jogo terminar, momento em que será apresentado o estado final e alguns arquivos com informações sobre a partida serão criados por meio de código.

Funcionamento Detalhado do Programa

¹ https://pt.wikipedia.org/wiki/Space_Invaders

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

A execução do programa será feita através da linha de comando (i.e., pelo *cmd*, *console*, ou terminal) e permitirá a passagem de parâmetros. As funcionalidades a serem realizadas pelo programa são descritas a seguir: *inicializar jogo*, *realizar jogo* e *gerar mapa de calor*. A descrição dos parâmetros de inicialização, da exibição do estado atual do programa, assim como, das operações a serem realizadas pelo programa, são apresentadas em detalhes a seguir.

Inicializar jogo: Para garantir o funcionamento correto do programa, o usuário deverá informar, ao executar o programa pela linha de comando, o caminho do diretório contendo os arquivos a serem processados (exemplo assumindo um programa compilado para o executável *prog*, “*./prog /maquinadoprofessor/diretoriodeteste*”). Considere um caminho com tamanho máximo de 1000 caracteres. O programa deverá verificar a presença desse parâmetro informado na linha de comando e, caso o usuário tenha esquecido de informar o nome do diretório, o programa deverá exibir uma mensagem de erro (ex. “ERRO: Informe o diretório com os arquivos de configuracao.”), finalizando sua execução. Dentro desse diretório, espera-se encontrar dois arquivos, um com as configurações iniciais do jogo (*config_inicial.txt*) e outro com o desenho dos personagens do jogo (*personagens.txt*). O programa deverá ler ambos os arquivos e preparar o ambiente de jogo. Caso o programa não consiga ler um deles (e.g., porque alguns deles não existem), ele deverá ser finalizado e imprimir uma mensagem informando que não conseguiu ler os arquivos (OBS: a mensagem deve conter o caminho e nome do arquivo que ele tentou ler). Todas as saídas em forma de arquivo do trabalho devem ser escritas na pasta “*saida*” dentro do mesmo diretório fornecido. Considerar que a pasta “*saida*” já estará criada nos casos de teste.

Abaixo é apresentado um modelo de como o *config_inicial.txt* é escrito:

config_inicial.txt

```
flag_seletor_funcionalidade
num_aliens tipo_alien_fileira_1 tipo_alien_fileira_2
```

O mapa terá sempre tamanho de 11 linhas e 21 colunas. Conforme visto no modelo acima, o arquivo *config_inicial.txt* apresenta várias informações conforme definido a seguir:

- Na primeira linha, o inteiro *flag_seletor_funcionalidade* define uma determinada funcionalidade que deve estar ativa. Ela poderá assumir vários valores, a depender do caso de teste, sendo que o valor determinará se uma determinada funcionalidade do jogo será ativada ou não. No momento, apenas dois valores estão definidos, 0 e 1. Outros valores desta flag serão definidos nas funcionalidades extras em sala. Caso esta flag seja 1, o arquivo *arquivo_saida.txt* (descrito na seção *gerar mapa de calor*) deverá conter o mapa de calor. Caso ela seja 0, esse arquivo deverá conter apenas a mensagem “Ainda a ser implementado!”. Isso permite que o aluno aprenda como inserir novas funcionalidades em sala.

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

- Na segunda linha, um inteiro *num_aliens*, representando o número de inimigos por fileira. O mapa deve conter 2 fileiras de inimigos. A primeira fileira de inimigos deve começar do ponto esquerdo superior do mapa (crescendo verticalmente para baixo e horizontalmente para a direita), com cada inimigo sendo separado por 1 espaço em branco entre eles (com $num_aliens \leq 4$). As fileiras devem ter um espaçamento de uma linha. O arquivo *personagens.txt* (explicado posteriormente neste documento) conterá 2 *sprites* de naves alienígenas. Os valores *tipo_alien_fileira_1* e *tipo_alien_fileira_2* correspondem ao tipo de alien (referente a qual *sprite*) que deverá ser desenhado para a primeira e na segunda fileira de inimigos, respectivamente.

Um exemplo desse arquivo de definição do mapa é mostrado a seguir.

config_inicial.txt

```
0
3 1 2
```

No exemplo acima, a primeira linha indica que o mapa de calor não deve ser impresso no arquivo de saída, mas apenas a mensagem definida acima. A segunda linha indica que o mapa deve conter 3 inimigos em cada fileira, totalizando 6 inimigos. Os dois números seguintes indicam que os alienígenas da primeira fileira de inimigos (começando do topo) devem apresentar o desenho do *sprite* 1, da segunda linha, do *sprite* 2.

É importante destacar que não haverá sobreposição entre as naves e as coordenadas dos arquivos consideram a posição (0, 0) como o canto superior esquerdo do mapa (sem as bordas de estilo).

O arquivo *personagens.txt* define os desenhos dos personagens do jogo, primeiramente o desenho do tiro (sempre 1x1) e da nave do jogador e, em seguida, o desenho do tiro (sempre 1x1) dos inimigos e 2 desenhos de naves alienígenas. O tamanho das naves é fixo, com 3 caracteres de largura e 2 caracteres de altura. Seguindo essa definição, a primeira linha do arquivo apresenta o caractere que define o tiro do jogador. Em seguida, sua representação em caracteres ocupando 2 linhas com 3 caracteres cada. A linha seguinte apresenta o caractere que representa o tiro dos inimigos. Nas linhas seguintes, são apresentados 2 desenhos de naves de inimigos, ambas com tamanho 3x2.

Um exemplo desse arquivo de definição dos personagens é mostrado a seguir.

personagens.txt

```
^
/i \
***
O
 V
\M/
=O=
\W/
```

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		



Neste arquivo de exemplo, o tiro do jogador é representado com o caractere “^” e o jogador é representado como abaixo:

Desenho do jogador:

```
/i\  
***
```

Já o tiro das naves alienígenas é representado com o caractere “O”. Dois desenhos de naves alienígenas são apresentados:

Desenho do alien (sprite 1):

```
v  
M
```

Desenho do alien (sprite 2):

```
=O=  
w
```

É importante frisar que a posição central (citada anteriormente) de todos os personagens é o caractere do meio da primeira linha do desenho (ou seja, meio do topo).

Ao inicializar o jogo, a base da nave do jogador deve ser posicionada na base do mapa e esta deve estar centralizada horizontalmente no mapa (posição central do personagem em $x=10$ considerando x começando em $x=0$ e terminando em $x=20$).

Realizar jogo: Uma vez que o jogo esteja preparado, o mapa da iteração 0 deve ser impresso (conforme descrito abaixo na última etapa realizada para cada iteração) e as jogadas podem começar a ser processadas. O jogo terminará quando o jogador eliminar todos os alienígenas do jogo ou morrer.

As jogadas deverão ser lidas da entrada padrão, permitindo o redirecionamento a partir de um arquivo (um arquivo entrada.txt será fornecido para cada caso de teste), ou seja, conforme feito nos exercícios do BOCA e sem precisar abrir o arquivo em código. Cada iteração de uma partida deve seguir, necessariamente na mesma ordem, as seguintes etapas:

- **Verificar condição de fim de jogo:** Verifique se o jogador eliminou todas as naves alienígenas (vitória) ou se ele morreu (derrota);
- **Ler entrada do usuário:** Leia a jogada fornecida pelo usuário no formato “@”, onde “@” é um caractere que define o movimento, podendo ser:
 - “a” para mover para a esquerda;
 - “d” para mover para a direita;
 - “ ” (espaço), o jogador deve tentar atirar;
 - “s” para ficar parado nesta rodada.

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

- Realizar ação do usuário:** Realize o movimento do jogador conforme o comando recebido: alterar a posição na horizontal, atirar ou não fazer nada. Caso o jogador dê o comando para atirar enquanto ainda houver um tiro dele na tela, o jogo deve consumir a entrada contando como uma iteração, porém deve interpretá-la como 'ficar parado'. Os tiros se iniciam sempre no centro horizontal no topo da nave do jogador (permitindo que posteriormente, durante a atualização do movimento dos tiros desta iteração, eles sejam movidos para a coordenada vertical logo acima).
- Mover naves alienígenas:** O primeiro movimento das naves é sempre para a direita, uma vez que elas se inicializam coladas na parede esquerda. O movimento deve ser sempre de 1 espaço na direção atual, até colidir com uma parede e mudar de direção. O movimento ocorre apenas em iterações pares do jogo. Todas as naves navegam sempre no mesmo sentido. A colisão com a parede ocorrerá caso a nave alienígena na posição mais externa (i.e. a nave mais a direita se o movimento for para a direita) encoste na borda do mapa. Abaixo, é apresentada uma sequência de telas que exemplificam esse comportamento. Observe que as naves alienígenas inicialmente estão se movendo para a direita (em iterações pares). Quando a nave alienígena mais à direita encosta na borda direita do mapa, o sentido da trajetória de todas as naves é mudada, indo para a esquerda no último quadro.

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

Pontos: 1 Iteracoes: 10 <pre> xMx xMx xMx xMx \X/ \X/ \X/ \X/ V V V V \M/ \M/ \M/ \M/ *o* <=> </pre>
Pontos: 1 Iteracoes: 11 <pre> xMx xMx xMx xMx \X/ \X/ \X/ \X/ V V V V \M/ \M/ \M/ \M/ *o* <=> </pre>
Pontos: 1 Iteracoes: 12 <pre> xMx xMx xMx xMx \X/ \X/ \X/ \X/ V V V V \M/ \M/ \M/ \M/ *o* <=> </pre>
Pontos: 1 Iteracoes: 13 <pre> xMx xMx xMx xMx \X/ \X/ \X/ \X/ V V V V \M/ \M/ \M/ \M/ *o* <=> </pre>

- Tiro inimigo:** Os tiros dos inimigos ocorrerão em iterações ímpares e múltiplas de 3 (3, 9, 15, 21...). Devem ter somente 3 tiros concomitantemente na tela, logo, se for uma iteração ímpar e múltipla de 3 e já existirem 3 tiros na tela, nenhuma nave deve atirar. A nave inimiga que deve atirar deve ser escolhida com base na posição do jogador, logo, a nave alienígena viva da fileira mais próxima ao jogador, cuja distância horizontal seja a menor para o jogador (com preferência para a nave da esquerda, em caso de empate). Abaixo, é apresentada uma sequência de telas que exemplificam esse comportamento. Observe que os tiros iniciam-se sempre no centro horizontal na base das naves alienígenas (permitindo que no próximo item eles sejam movidos para a coordenada vertical logo abaixo). Note ainda que a nave escolhida está na fileira de inimigos mais próxima do jogador e é a nave mais próxima dele. Note também que o tiro foi dado durante a iteração 3 (só aparecendo,

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

logo abaixo, na iteração 4) e não acontece na iteração 6 (múltiplo de 3, porém par) e nem na 7 (ímpar, porém não múltiplo de 3). O próximo tiro seria dado na iteração 9.

Pontos: 0 Iteracoes: 3 xMx xMx xMx xMx \X/ \X/ \X/ \X/ V V V V \M/ \M/ \M/ \M/ *o* <=>
Pontos: 0 Iteracoes: 4 xMx xMx xMx xMx \X/ \X/ \X/ \X/ V V V V \M/ \M/ \M/ \M/ *o* <=>
Pontos: 0 Iteracoes: 5 xMx xMx xMx xMx \X/ \X/ \X/ \X/ V V V V \M/ \M/ \M/ \M/ * *o* <=>
Pontos: 0 Iteracoes: 6 xMx xMx xMx xMx \X/ \X/ \X/ \X/ V V V V \M/ \M/ \M/ \M/ * *o* <=>

- **Atualizar movimento dos tiros:** Atualize a posição dos tiros aliados (movem-se para cima no mapa) e dos tiros inimigos (movem-se para baixo no mapa). Em seguida, o mapa de calor é atualizado com as novas posições dos tiros.
- **Verificar colisão entre tiros aliados e naves alienígenas:** Verifique se houve colisão entre os tiros do jogador e alguma nave alienígena. Em caso positivo, tal nave deve perder uma vida (toda nave alienígena será inicializada com duas vidas) e o jogador deve receber 1 ponto (se ela continuar viva). Se as vidas de tal nave chegarem a zero, ela morrerá devendo sumir da tela e dando 3 pontos para o jogador. Obviamente, naves mortas não influenciam mais no comportamento do

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

jogo (por exemplo, não atiram e não afetam os cálculos de direção de trajetória das fileiras de inimigos).

- **Verificar condições de fim de jogo:** Verifique se o jogador eliminou todas as naves alienígenas. Caso negativo, verifique se houve colisão entre os tiros das naves alienígenas e o jogador, se houver colisão, a nave do jogador deve desaparecer da tela. Se uma das condições acima for verdadeira, o jogo deve ser finalizado (após executar os passos seguintes). No caso das verificações de fim de jogo (vitória ou derrota), deve ser exibida a mensagem “Parabens! Voce eliminou todas as naves alienigenas e venceu!” em caso de vitória, e “As naves alienigenas te eliminaram! Fim de jogo.” em caso de derrota;
- **Incrementar contador de iterações:** Incrementar o contador de iterações da partida (que inicia-se na iteração 0).
- **Atualizar e imprimir mapa:** Exibir na saída padrão (ou seja, na tela) o estado atual do jogo na seguinte ordem: primeiro os tiros aliados, depois os tiros inimigos, a nave do jogador e, por fim, as naves alienígenas. Essa ordem de impressão definirá o elemento que aparecerá na tela em caso de sobreposição (o último elemento impresso será o visível). Além disso, devem ser mostradas a pontuação acumulada até o momento e o número da iteração atual, no seguinte formato:

```

Pontos: # | Iteracoes: @
+-----+
| xMx  xMx  xMx  xMx |
| \X/  \X/  \X/  \X/ |
|      |      |      |
|  V    V    V    V  |
| \M/  \M/  \M/  \M/ |
|      |      |      |
|      * |          |
|      *o*         |
|      <=>          |
+-----+

```

Onde # e @ podem assumir o valor de um número inteiro positivo qualquer. Abaixo é apresentado um exemplo de uma tela:

```

Pontos: 0 | Iteracoes: 6
+-----+
| xMx  xMx  xMx  xMx |
| \X/  \X/  \X/  \X/ |
|      |      |      |
|  V    V    V    V  |
| \M/  \M/  \M/  \M/ |
|      |      |      |
|      * |          |
|      *o*         |
|      <=>          |
+-----+

```

Para facilitar a implementação, será fornecido também, com cada caso de teste, um arquivo de movimentos (denominado entrada.txt), que poderá ser redirecionado pela

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

entrada padrão para gerar uma saída esperada. Isso evitará ter que digitar todas as jogadas na mão. Veja abaixo um exemplo do conteúdo de um arquivo de movimentos. Considere que os casos dados sempre serão suficientes para finalizar o jogo, com uma vitória ou uma derrota. Cabe ao aluno testar outros casos.

entrada.txt

```
d
a
d
a

a
d

d
d
a
s
```

Gerar Mapa de Calor: Ao final da execução do jogo, o programa deve gerar um arquivo chamado `arquivo_saida.txt`, localizado na pasta de saída referente ao caso de teste em execução. Se a flag `flag_seletor_funcionalidade` for 1, este arquivo deve apresentar um mapa de calor que contabiliza a quantidade de vezes que um tiro (do jogador ou inimigo) passou por cada célula do mapa. Os valores registrados no mapa de calor não devem ultrapassar o limite de 99; caso esse valor seja excedido, ele deve ser mantido forçadamente em 99. Para facilitar a visualização do arquivo, cada número no mapa deve ser formatado com exatamente dois caracteres de largura, utilizando a notação `"%02d"` e devem ser separados por espaço.

Um exemplo de arquivo pode ser visto abaixo:

arquivo_saida.txt de exemplo

```
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 01 00
00 00 01 00 00 00 01 00 00 00 01 00 00 00 01 01 01 00 00 01 00
00 01 01 00 01 00 01 00 00 00 01 02 00 01 03 01 00 00 01 00
00 01 01 00 01 00 01 00 00 00 01 02 00 01 03 01 00 00 01 00
00 01 01 00 01 00 01 00 00 00 01 02 00 01 03 01 00 00 01 00
00 01 00 00 01 00 00 00 00 00 02 00 01 02 00 00 00 00 00 00
00 00 00 00 01 00 00 00 00 00 02 00 01 02 00 00 00 00 00 00
```

Informações Gerais

Alguns arquivos de entrada e respectivos arquivos de saída serão fornecidos para o aluno. O aluno deverá utilizar tais arquivos para testes durante a implementação do trabalho. É de responsabilidade do aluno criar novos arquivos para testar outras possibilidades do

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

programa e garantir seu correto funcionamento. O trabalho será corrigido usando, além dos arquivos dados, outros arquivos (específicos para a correção e não disponibilizados para os alunos) seguindo a formatação descrita neste documento. Em caso de dúvida, pergunte ao professor. O uso de arquivos com formatação diferente poderá acarretar na incompatibilidade durante a correção do trabalho e consequentemente na impossibilidade de sua correção (sendo atribuído a nota zero). Portanto, siga estritamente o formato estabelecido e use o script a ser fornecido com o trabalho para testá-lo.

Implementação e Verificação dos Resultados

A implementação deverá seguir os conceitos de modularização vistos em sala. O trabalho terá uma componente subjetiva que será avaliada pelo professor para verificar o grau de uso dos conceitos ensinados. Portanto, além de funcionar corretamente, o código deverá estar bem escrito para que o aluno obtenha nota máxima.

É extremamente recomendado (para não dizer obrigatório) utilizar algum programa para fazer as comparações do resultado final do programa. Isto é, os arquivos de saída gerados, poderão ser comparados com os arquivos de saídas esperadas (fornecidos pelo professor) utilizando o comando *diff*, como visto e feito em sala. O *Meld* é uma alternativa gráfica para o *diff*, se você preferir. Esse programa faz uma comparação linha a linha do conteúdo de 2 arquivos e é muito útil no desenvolvimento do trabalho. Diferenças na formatação poderão impossibilitar a comparação e consequentemente impossibilitar a correção do trabalho. O programa será considerado correto se gerar a saída esperada idêntica à fornecida com os casos de teste.

O trabalho será corrigido com um script de correção automática (a ser fornecido com o trabalho). Os casos de testes visíveis serão disponibilizados juntamente com esta especificação do trabalho, sendo os casos ocultos disponibilizados em um momento posterior. A estrutura de diretórios e forma de execução do script de correção será disponibilizada em na documentação do script (um arquivo README.md) a ser fornecida juntamente com o trabalho. Um canal no *discord* será disponibilizado para tirar dúvidas sobre o trabalho e respectivo script.

Regras Gerais

O trabalho deverá ser feito individualmente e pelo próprio aluno (sem ajuda de terceiros ou uso de IA para geração de código). O aluno deverá necessariamente conhecer e dominar todos os trechos de código criados, além de dominar o uso do script que fará a correção. O aluno não deve usar recursos de programação não ensinados na disciplina (por exemplo,

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

alocação dinâmica com ponteiros). Cada aluno deverá trabalhar independente dos outros, não sendo permitido a cópia ou compartilhamento de código. O professor irá fazer verificação de plágio. Trabalhos identificados como iguais, em termos de programação (por exemplo, mudar nomes de variáveis e funções entre outros não faz dois trabalhos serem diferentes), serão penalizados com a nota zero e submetidos para penalidades adicionais em instâncias superiores. Isso também inclui a pessoa que forneceu o trabalho, sendo portanto, de sua obrigação a proteção de seu trabalho contra cópias ilícitas. Proteja seu trabalho e não esqueça cópias do seu código nas máquinas de uso comum. Caso necessário, o professor poderá realizar uma entrevista para verificar o domínio sobre o código.

Entrega do Trabalho

Instruções de entrega do trabalho serão passadas em sala e pelo classroom, devendo as entregas respeitarem a data e hora limite definidas para tal. O trabalho deve ser entregue todo em um único arquivo “.c”, nomeado com o nome do aluno, sem espaços e sem acentos, por exemplo “ThiagoOliveiraDosSantos.c”. Ele será necessariamente corrigido no sistema operacional linux das máquinas do laboratório, qualquer incompatibilidade devido ao desenvolvimento em um sistema operacional diferente é de responsabilidade do aluno. Isso pode inclusive levar a nota zero, no caso de impossibilidade de correção. Faça backups constantes do seu trabalho, uma vez que não é raro perder tudo perto da data de entrega. É responsabilidade do aluno ter cópias de segurança na nuvem. A correção será feita de forma automática (via script), portanto trabalhos não respeitando as regras de geração dos arquivos de saída, ou seja, fora do padrão, poderão impossibilitar a correção. Consequentemente, isso acarretará na atribuição da nota zero. A pessoa corrigindo não terá a obrigação de adivinhar nome de arquivos, diretórios ou outros. **Siga estritamente o padrão estabelecido!** O script já dá uma prévia da nota que o aluno receberá.

Pontuação e Processo de Correção

Pontuação: Trabalhos entregues após o prazo não serão corrigidos (recebendo a nota zero). O trabalho será pontuado de acordo com sua implementação e a tabela abaixo. Os pontos descritos na tabela não são independentes entre si, isto é, alguns itens são pré-requisitos para obtenção da pontuação dos outros. Por exemplo, gerar o arquivo de mapa de calor e manutenção a ser feita em sala dependem de realizar o jogo corretamente. Código com falta de legibilidade e modularização pode perder pontos conforme informado na tabela. Erros gerais de funcionamento, lógica ou outros serão descontados como um todo. A

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

pontuação de um item será dada pelo número de casos de testes corretos para aquele item.

Percebam que no melhor dos casos os pontos da tabela abaixo somam 12 ao invés de 10. Isso foi feito propositalmente para ajudar os alunos esforçados com dois pontos extras. Esses pontos, caso obtidos, irão complementar uma das notas, do trabalho ou das provas parciais do semestre.

Item	Tur.	Quesitos	Ponto
Realizar jogo	PE	Receber corretamente as entradas do jogador Mostrar as informações do jogo como especificado Mostrar o resultado do jogo Usar a saída padrão	0,8
Gerar mapa de calor	PE	Gerar corretamente o arquivo com o mapa de calor (arquivo_saida.txt)	0,2
Manutenibilidade 1	TM	Implementação de modificações surpresas e serem passadas em sala de aula no primeiro dia de correção do trabalho.	5,5
Manutenibilidade 2	TM	Implementação de modificações surpresas e serem passadas em sala de aula no segundo dia de correção do trabalho.	5,5
Legibilidade e Modularização	PE	Falta de: • Uso de comentários; • Identação do código; • Uso de funções; • Uso de tipos de dados definidos pelo usuário	-1

Processo de correção do trabalho: O trabalho será corrigido, considerando-se 3 turnos: Primeira Entrega (PE), Tempo de Manutenção (TM) e Entrevista (sob demanda).

	Centro Tecnológico Departamento de Informática	
Disciplina: Programação I		Código: INF15927
Trabalho Prático		

A Primeira Entrega ocorrerá na data definida para a Entrega do Trabalho. Ela gerará uma nota da Primeira Entrega que servirá como parte da nota final do trabalho (ver pontos referentes à Primeira Entrega na tabela de pontuação, marcado com PE).

O Tempo de Manutenção ocorrerá, em sala, nos dias indicados no classroom. Nesses dias, o aluno terá a possibilidade, dentro do tempo disponibilizado, de corrigir erros encontrados no trabalho devendo também implementar novas funcionalidades a serem fornecidas no dia. A manutenibilidade do código será avaliada com a implementação de novas funcionalidades para incorporação de pequenas mudanças no comportamento do jogo. Isso faz parte da pontuação do trabalho (ver pontos referentes à Tempo de Manutenção na tabela de pontuação, marcado com TM). Quanto mais modular estiver o código, mais fácil será de implementar as mudanças. Como se pode ver na tabela, grande parte da nota do trabalho (90%) vem da implementação de funcionalidades extras em sala. Portanto, é fundamental que o aluno tenha conhecimento e domine todo o código de seu trabalho, além do uso do script de correção.

A Entrevista ocorrerá em uma data posterior a aula de correção e fora do horário de aula (a ser agendado). Ela tem o intuito de levantar o conhecimento dos alunos sobre o próprio trabalho quando necessário, podendo apenas reduzir sua nota caso não demonstre conhecimento sobre o próprio código. Portanto, ela será feita sob demanda para avaliar o conhecimento do aluno a respeito do seu próprio programa.

Considerações Finais

Não utilizar caracteres fora da tabela ASCII em lugar nenhum do trabalho (por exemplo, letras acentuadas).